INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

SBT-DF204: Computer Forensics Case Studies

Case Study 1: Investigating Harassment Email Traffic With Wireshark

Individual Forensic Investigation Report

Field	Detail
Student Name	Mary-Joy Adewole
Registration Number	2025/FWSD/11468
Institution	International Cybersecurity and Digital Forensics Academy (ICDFA)
Course Code	SBT-DF204
Course Title	Computer Forensics Case Studies
Assessment	Case Study 1: Individual forensic investigation
Case Title	Investigating Harassment Email Traffic With Wireshark
Evidence File	nitroba.pcap (2008 Nitroba University Harassment Scenario, Digital Corpora)
Analysis Date	3 October 2026
Tools	Wireshark and TShark/capinfos on Kali Linux 2026.2 (VMware Workstation); Firefox PDF viewer for scenario slides
Time Basis	All packet times are UTC, as displayed by Wireshark (View > Time Display Format > UTC Date and Time of Day)

1. Case Reference and Student Identification

This report is the individual submission of Mary-Joy Adewole (registration number 2025/FWSD/11468) for SBT-DF204 Case Study 1. It examines a simulated harassment-email complaint made by Lily Tuckrige, a Chemistry Department teacher at the fictional Nitroba State University, and assesses whether the supplied network capture supports attribution of a harassing message sent through willselfdestruct.com to a student in Chemistry 109.

Handling and scope. The capture was treated as controlled training material and used only for this authorised academic case study. Analysis was performed offline on a working copy. No host, account or person referenced in the evidence was contacted, and no connection was made to willselfdestruct.com or any address in the capture. The only network downloads were the public scenario files (the PCAP and the scenario slide deck) from Digital Corpora. Unrelated personal data in screenshots was redacted (see Appendix C). All people, accounts and organisations are part of a fictional scenario.

2. Executive Conclusion

The capture shows that one device on the residence network (Ethernet MAC 00:17:f2:e2:c0:ce, using private address 192.168.15.4) loaded the willselfdestruct.com submission form (packet 82936, 2008-07-22 06:03:43.825871 UTC) and then sent the only HTTP POST to that service in the capture (packet 83601, 06:04:24.311700 UTC). The POST's recipient (lilytuckrige@yahoo.com), subject ("you can't find us"), message text and 30-second lifetime correspond to the second harassing email shown in the scenario slides.

Before and after that POST, the same device sent 100 requests to mail.google.com carrying a cookie naming the account jcoachj@gmail.com (packets 78967 to 84201). The Chemistry 109 roster contains one entry whose surname matches that account name: Johnny Coach. The roster lists names only, so this link is a name-pattern inference.

The evidence therefore supports the conclusion that a computer in the shared dorm room, on which an account matching a Chem 109 roster entry was in use, sent the message. It does not prove who operated the computer. The room's Wi-Fi router had no password and three residents share the room, the willselfdestruct request itself carries no identifying cookie, and the sender field was left blank. Confidence is moderate-to-high that this device sent the message, moderate that the holder of the jcoachj account was using it, and low-to-moderate that the account holder personally sent the message.

3. Evidence Acquisition and Integrity

The capture was downloaded once, hashed, write-protected and then copied. All analysis used the copy. The record below is taken from the terminal output in Figures 1 to 5.

Attribute	Value
Original file name	nitroba.pcap
Source link	https://digitalcorpora.s3.amazonaws.com/corpora/scenarios/2008-nitroba/nitroba.pcap (linked from the Digital Corpora scenario page and the assessment brief)
File size	56,180,821 bytes (wget reported 56180821/56180821 saved)
Download date/time	Saturday 3 October 2026, 16:14:39 UTC (date -u). The wget log shows 12:14:39 because the Kali VM clock is UTC-4.
SHA-256	2b77a9eaefc1d6af163d1ba793c96dbccacb04e6befdf1a0b01f8c67553ec2fb
MD5	9981827f11968773ff815e39f5458ec8
SHA-1	65656392412add15f93f8585197a8998aaeb50a1
Published checksum comparison	The scenario page publishes MD5, SHA1 and SHA256 values for nitroba.pcap. All three match the calculated values (sha256sum -c, md5sum -c and sha1sum -c each returned OK).
Working copy	cp -p nitroba.pcap ../working/nitroba_working.pcap. SHA-256 of the copy is identical to the original ("HASHES MATCH").
Original protection	chmod a-w applied after hashing; ls -l afterwards shows -r--r--r--.
Capture properties (capinfos)	pcap, Ethernet, 94,410 packets, microsecond timestamps, 1 interface, strict time order True, duration 15,759.950751 s. First packet 2008-07-22 01:51:07.095278 UTC; last packet 2008-07-22 06:13:47.046029 UTC.

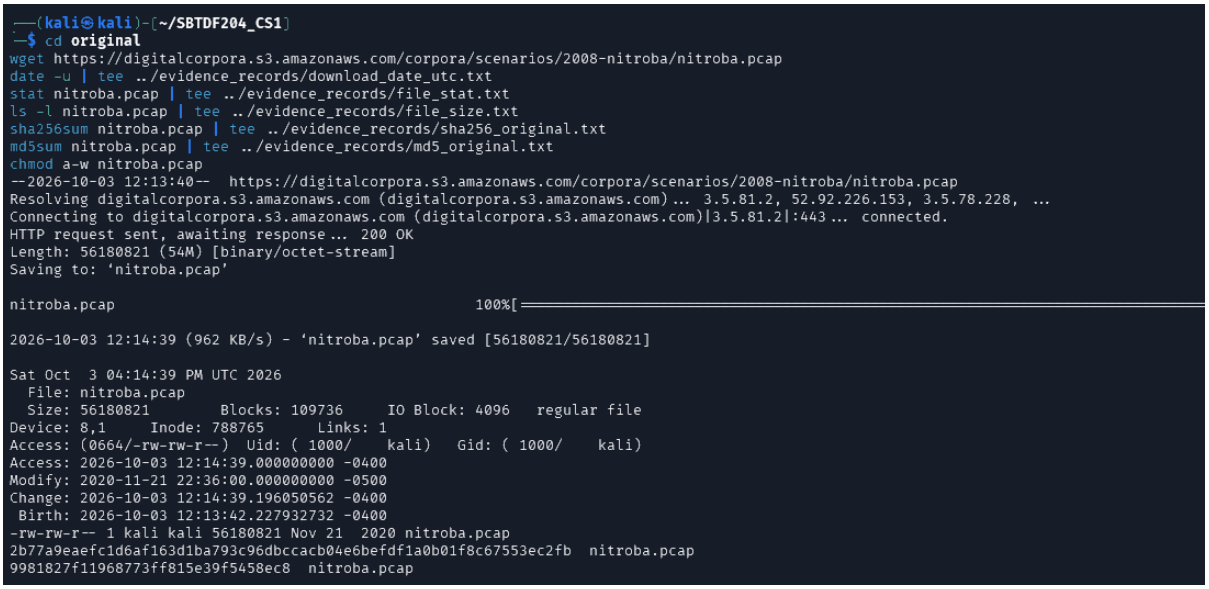


Figure 1: Download of nitroba.pcap (wget), download date in UTC, stat output, and SHA-256 and MD5 values of the original file. (E01)

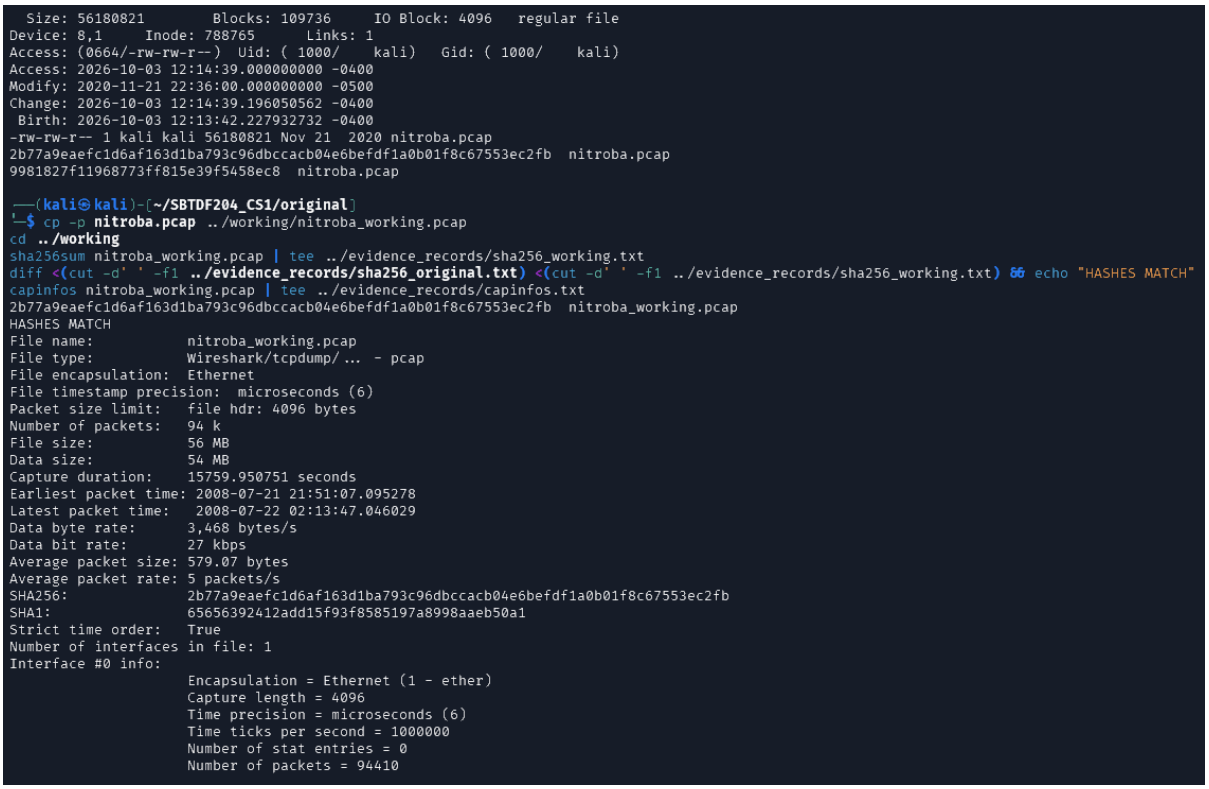


Figure 2: Working copy created with cp -p, hash comparison ("HASHES MATCH") and capinfos summary. Capinfos times here are shown in the VM's local time (UTC-4); see Figure 5 for the UTC values. (E02)

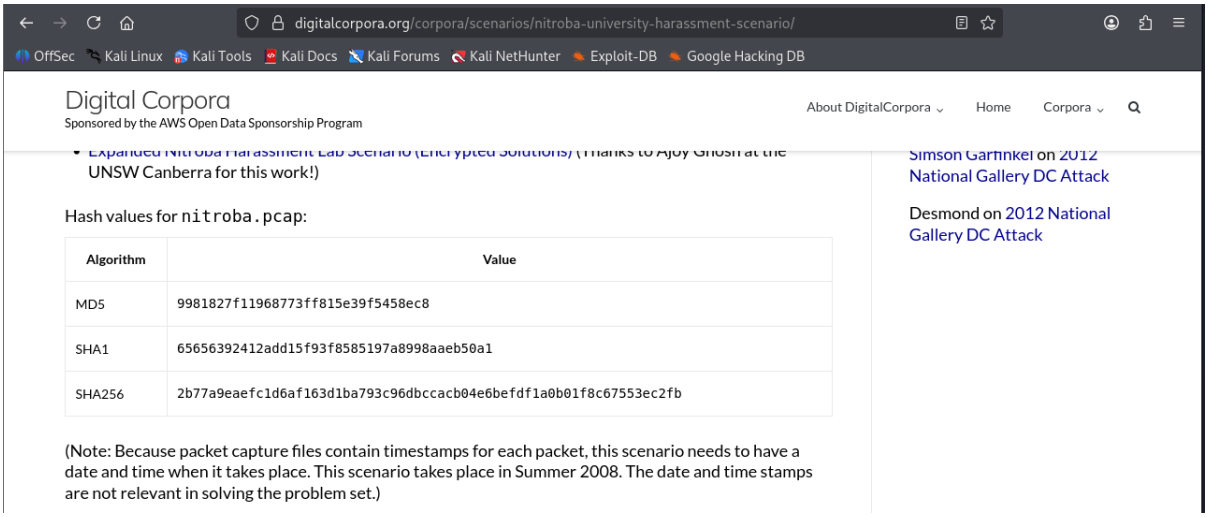


Figure 3: Published MD5, SHA1 and SHA256 values for nitroba.pcap on the Digital Corpora scenario page. (E01)



Figure 4: Published hashes saved to a text file and verified against the original with md5sum, sha1sum and sha256sum (all OK). (E01)

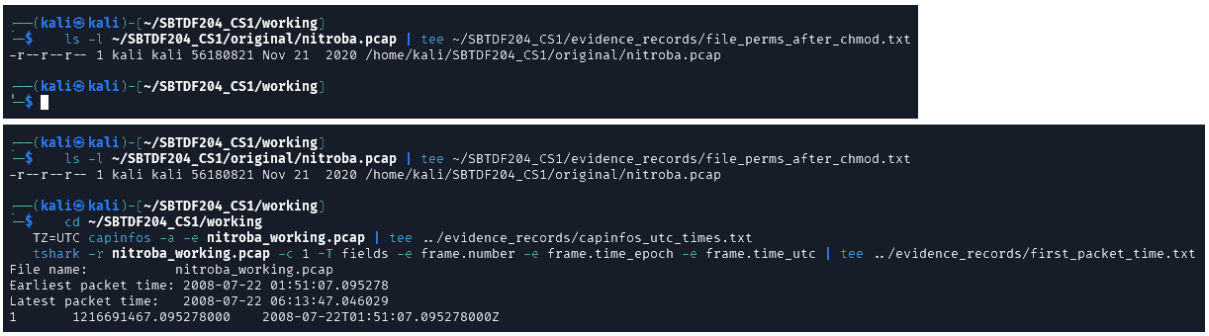


Figure 5: Original file permissions after chmod a-w (-r--r--r--), and capinfos/tshark output run with TZ=UTC showing the capture span in UTC (packet 1: 2008-07-22T01:51:07.095278Z). (E02)

4. Method

The review followed this sequence. Each step was checked against packet details before a conclusion was recorded. A filter result alone was not treated as a finding.

- Preserve and verify the evidence (Section 3), then open only the working copy in Wireshark and set the time display to UTC.
- Locate the web service and client using an HTTP GET filter on the host name, then inspect Ethernet, IP, TCP and HTTP layers of the first request.
- Isolate the submission with a POST filter limited to the client and service IPs, read the HTML form fields, and use Follow TCP Stream on the stream to see the request and server reply together. Confirm with tshark that the capture contains only one POST to the service.
- Trace the device by filtering on the client MAC, then check how many IP addresses that MAC uses and which MACs the client IP uses.

- Look for identity evidence with an @ filter on cookies sent from the client MAC, extract the identity-bearing cookie fragments, locate the packets nearest the POST, and compare User-Agent strings. Raw cookie dumps were kept out of the report because they contain unrelated session tokens.
- Compare the supported identity with the Chem 109 roster and compare the POST with the second email shown in the scenario slides.
- Build a UTC timeline from packet timestamps and separate observations, inferences and limitations.

Key display filters used (adapted from the brief's starting points after testing against the capture):

```
http.request.method == "GET" && http.host == "www.willselfdestruct.com"
ip.src == 192.168.15.4 && ip.dst == 69.25.94.22 && http.request.method == "POST"
tcp.stream eq 1707
eth.src == 00:17:f2:e2:c0:ce && http.request.method == "POST"
eth.src == 00:17:f2:e2:c0:ce && http.cookie contains "@"
frame.number == 83326
```

5. Findings

Each finding cites the filter or command, packet or stream, UTC timestamp, evidence-log ID and figure. Facts observed in the capture are separated from inferences. Scenario slides are cited by PDF page number of slides.pdf.

5.1 Question 1: What file did you acquire, and how did you preserve its integrity?

Display filter / command	Packet / stream	Timestamp (UTC)	Evidence ID	Figures
sha256sum, md5sum, stat, capinfos, chmod a-w	Whole file	Download 2026-10-03 16:14:39 UTC	E01, E02	Fig 1–5

I acquired nitroba.pcap (56,180,821 bytes) from the Digital Corpora link in the brief. I recorded its SHA-256, MD5 and SHA-1 values, confirmed that all three match the values published on the scenario page, made it read-only, and created a byte-identical working copy (nitroba_working.pcap) verified by hash. All findings below come from the working copy. Full details are in Section 3.

5.2 Question 2: Which client system contacted the web service?

Display filter / command	Packet / stream	Timestamp (UTC)	Evidence ID	Figures
http.request.method == "GET" && http.host == "www.willselfdestruct.com" (9 of 94,410 packets)	Packet 82936; TCP stream 1680	2008-07-22 06:03:43.825871	E03	Fig 6, 7

Direct observation: all nine matching requests run from 192.168.15.4 to 69.25.94.22. The first, packet 82936, is GET /secure/submit with the Host header www.willselfdestruct.com (source port 35984, destination port 80). The other eight are six image requests for the form page, GET /secure/success (packet 83614) and GET /images/bk-message.gif (packet 83654). The client is therefore 192.168.15.4 and the service address is 69.25.94.22. Frame details for packet 82936 show Arrival Time 02:03:43.825871 EDT and UTC Arrival Time 06:03:43.825871 UTC, which confirms the UTC display.

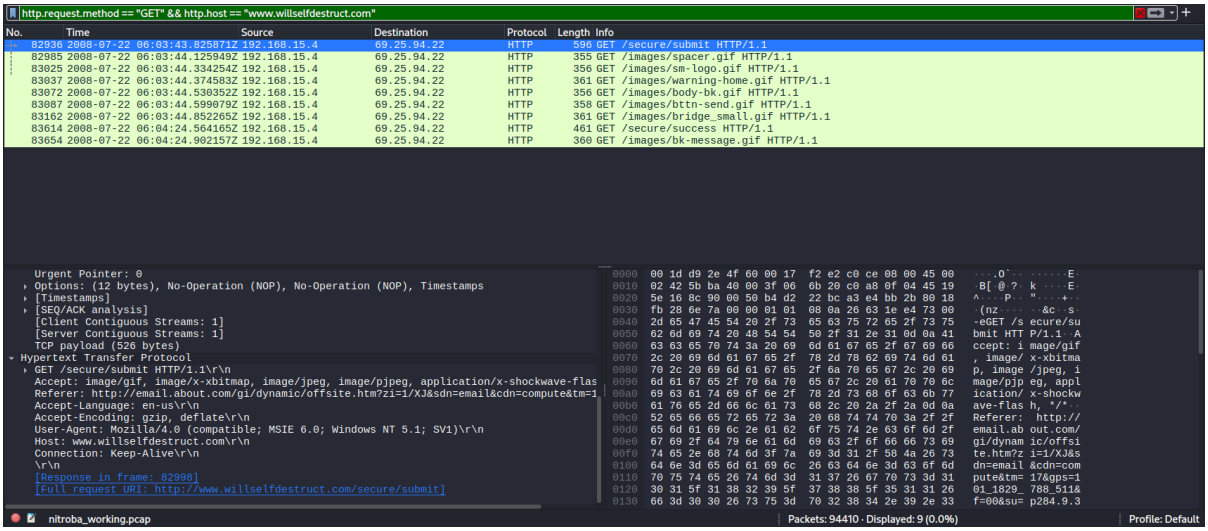


Figure 6: GET filter for www.willselfdestruct.com showing nine requests from 192.168.15.4 to 69.25.94.22, UTC time column, and HTTP details for packet 82936. (E03)

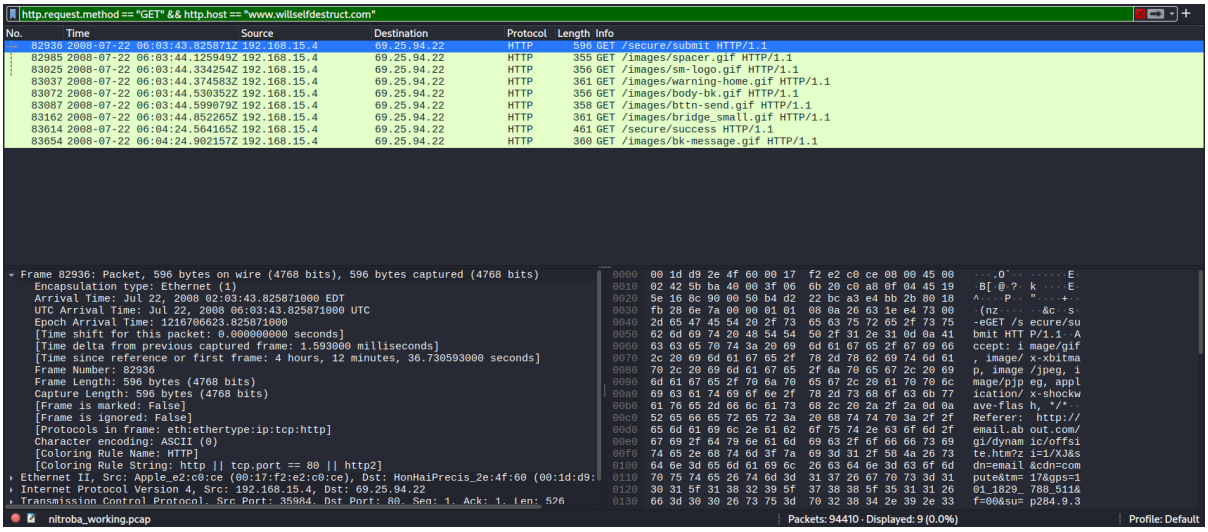


Figure 7: Frame details for packet 82936 showing local (EDT) and UTC arrival times, epoch 1216706623.825871. (E03)

5.3 Question 3: What evidence links the client to the harassment message?

Display filter / command	Packet / stream	Timestamp (UTC)	Evidence ID	Figures
ip.src == 192.168.15.4 && ip.dst == 69.25.94.22 && http.request.method == "POST"	Packet 83601; TCP stream 1707 (src port 36044)	2008-07-22 06:04:24.311700	E04, E06	Fig 8–11
tcp.stream eq 1707 (server reply)	Packets 83604 (302), 83614 (GET /secure/success)	06:04:24.469169; 06:04:24.564165	E05	Fig 9, 10

Direct observation: packet 83601 is POST /secure/submit (application/x-www-form-urlencoded, 188-byte body) from 192.168.15.4 to 69.25.94.22. Its form fields are to = lilytuckrige@yahoo.com, from = (blank), subject = “you can’t find us”, message = “and you can’t hide from us. Stop teaching. Start running.”, type = 0 and ttl = 30. The server answered in packet 83604 with 302 Moved Temporarily to /secure/success, and the client then requested /secure/success (packet 83614). A tshark export confirmed that this is the only POST to willselfdestruct.com in the whole capture (Figure 11).

The POST in packet 83601 corresponds to the second harassing email in the scenario slides:

Item	POST in capture (packet 83601)	Scenario slides (PDF page)	Match
Recipient	to=lilytuckrige@yahoo.com	To: lilytuckrige@yahoo.com (p.12); Yahoo mailbox of lilytuckrige (p.10)	Yes
Subject	you can't find us	Subject: you can't find us (p.10, p.12)	Yes
Message text	and you can't hide from us. / Stop teaching. / Start running.	Same three lines in the message box (p.12)	Yes
Lifetime	ttl=30	"only once for 30 second(s)" (p.11)	Yes
Sender	from= (blank)	From: [Undisclosed Sender] (p.12)	Consistent
Time	2008-07-22 06:04:24 UTC	Mon, 7/21/08 11:04 PM in the Yahoo inbox (p.10)	Consistent if mailbox clock is UTC-7 (inference)

Inference: the mailbox clock assumption comes from the first email's headers, where the Received line shows 17:21:01 -0700 and the mailbox lists the same message at 5:21 PM (PDF pages 3 and 5). Converting 11:04 PM at UTC-7 gives 06:04 UTC on 22 July, which matches the POST. This assumes the mailbox setting did not change between 13 and 21 July. The slides do not state a time zone.

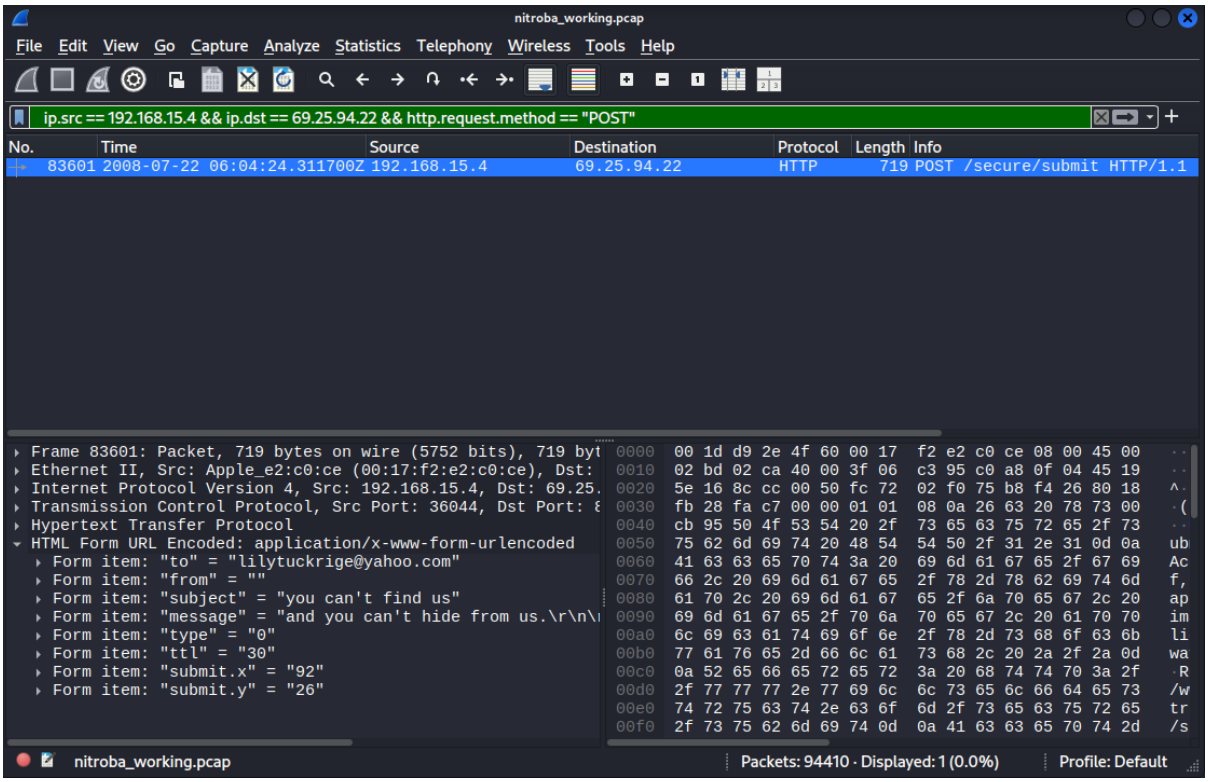


Figure 8: Filter isolating the single POST (packet 83601) with HTML Form URL Encoded fields: to, from, subject, message, type, ttl. (E04)

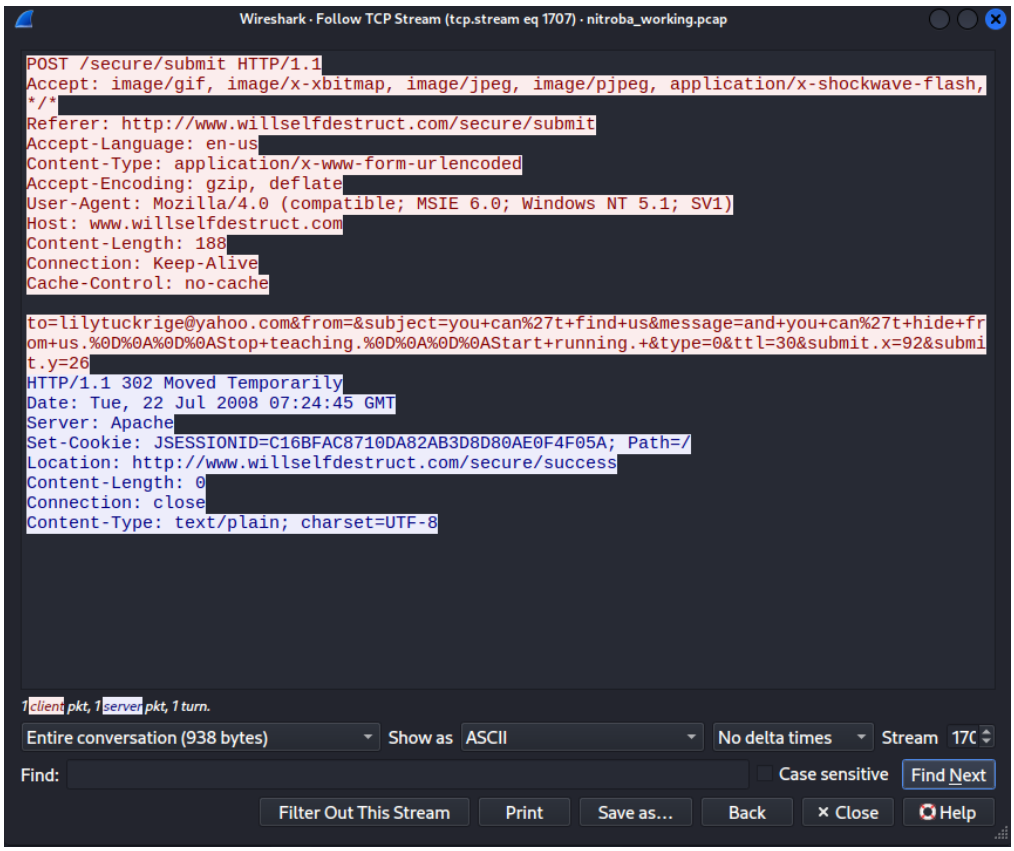


Figure 9: Follow TCP Stream (tcp.stream eq 1707): request with URL-encoded body and the 302 reply to /secure/success. The reply's Date header (07:24:45 GMT) differs from the capture time by about 1 h 20 min; see Section 7. (E04, E05)

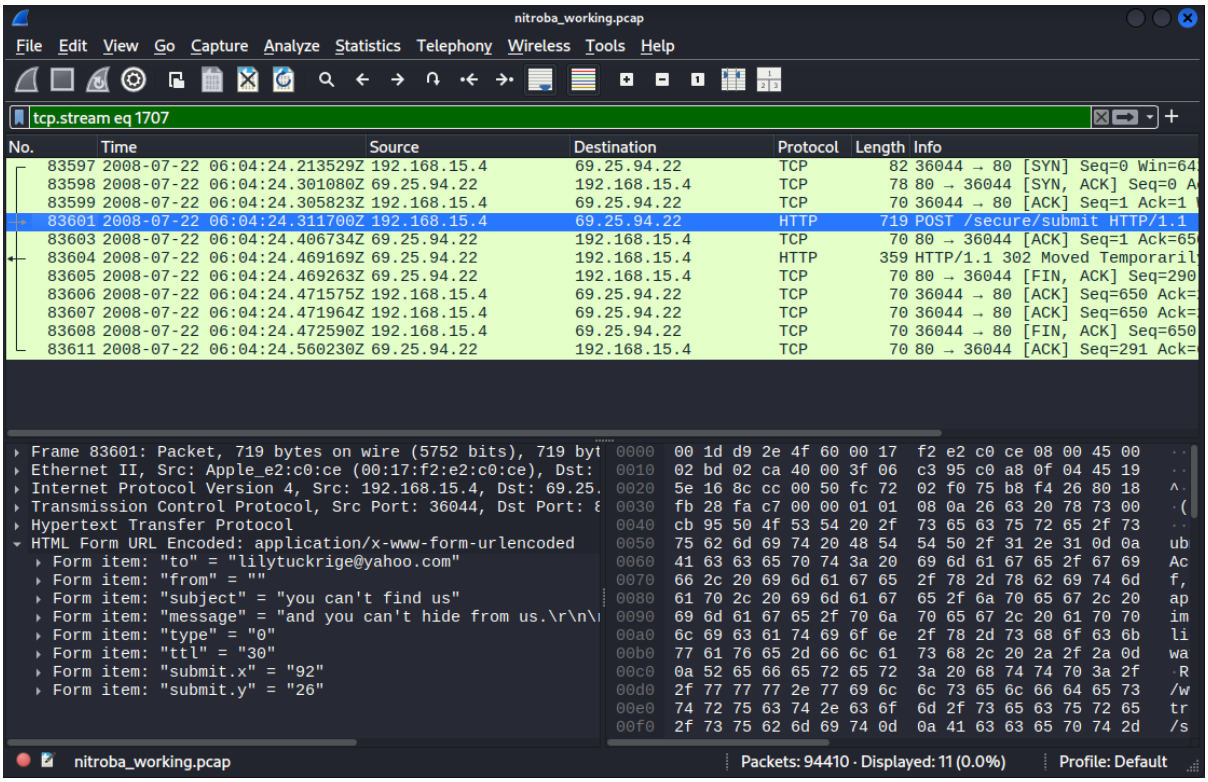


Figure 10: Stream 1707 packet list: handshake, POST (83601, 06:04:24.311700 UTC), 302 reply (83604) and connection close. (E04, E05)

```
┌──(kali㉿kali)-[~/SBTDF204_CS1]
└─$ cd ~/SBTDF204_CS1

tshark -r working/nitroba_working.pcap -Y 'http.request.method == "GET" && http.host contains "willselfdestruct"' \
  -T fields -e frame.number -e frame.time_utc -e frame.time_epoch -e eth.src -e ip.src -e ip.dst -e tcp.srcport -e tcp.stream -e http.request.uri \
  -E header=y -E separator=, > evidence_records/get_packets.csv

tshark -r working/nitroba_working.pcap -Y 'http.request.method == "POST" && http.host contains "willselfdestruct"' \
  -T fields -e frame.number -e frame.time_utc -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e ip.dst -e tcp.srcport -e tcp.stream -e http.request.uri \
  -E header=y -E separator=, > evidence_records/post_packets.csv

tshark -r working/nitroba_working.pcap -q -z follow,tcp,ascii,1707 > evidence_records/followstream_1707.txt
tshark -r working/nitroba_working.pcap -Y "frame.number == 83601" -V > evidence_records/frame_83601_detail.txt

wc -l evidence_records/post_packets.csv
cat evidence_records/post_packets.csv
2 evidence_records/post_packets.csv
frame.number,frame.time_utc,frame.time_epoch,eth.src,eth.dst,ip.src,ip.dst,tcp.srcport,tcp.stream,http.request.uri
83601,2008-07-22T06:04:24.311700000Z,1216706664.311700000,00:17:f2:e2:c0:ce,00:1d:d9:2e:4f:60,192.168.15.4,69.25.94.22,36044,1707,/secure/submit

┌──(kali㉿kali)-[~/SBTDF204_CS1]
└─$
```

Figure 11: TShark export showing exactly one POST to willselfdestruct.com (header plus one row) with frame number, UTC time, epoch time, MACs, IPs, port, stream and URI. (E06)

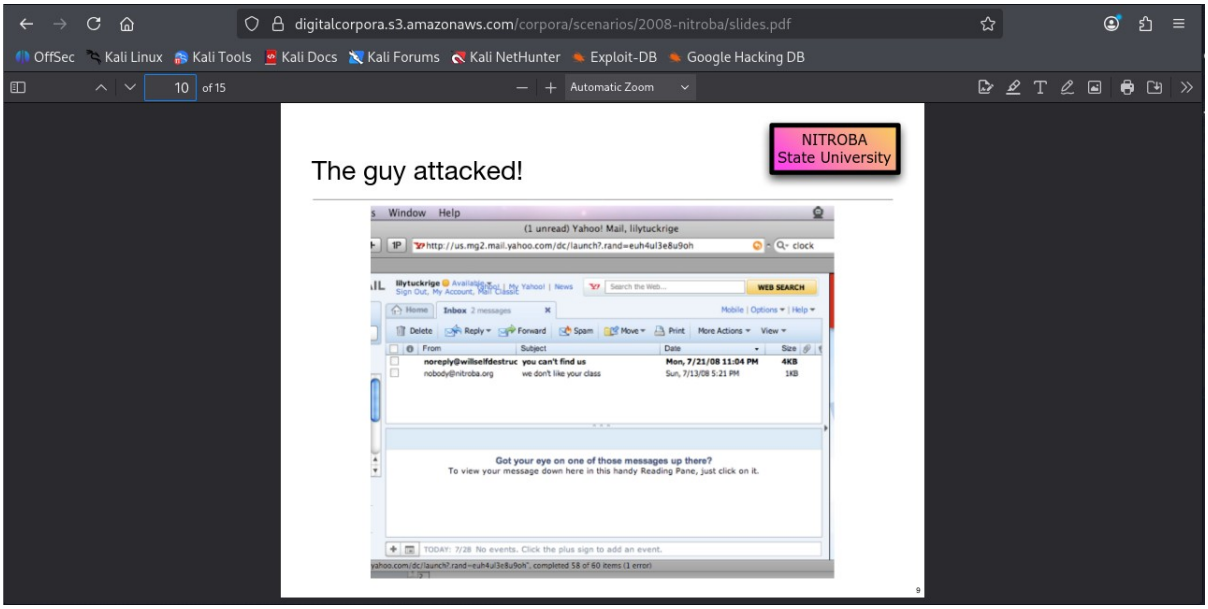


Figure 12: Scenario slide (PDF page 10): Yahoo inbox showing a message from noreply@willselfdestruct… with subject "you can't find us", Mon 7/21/08 11:04 PM. (E14)

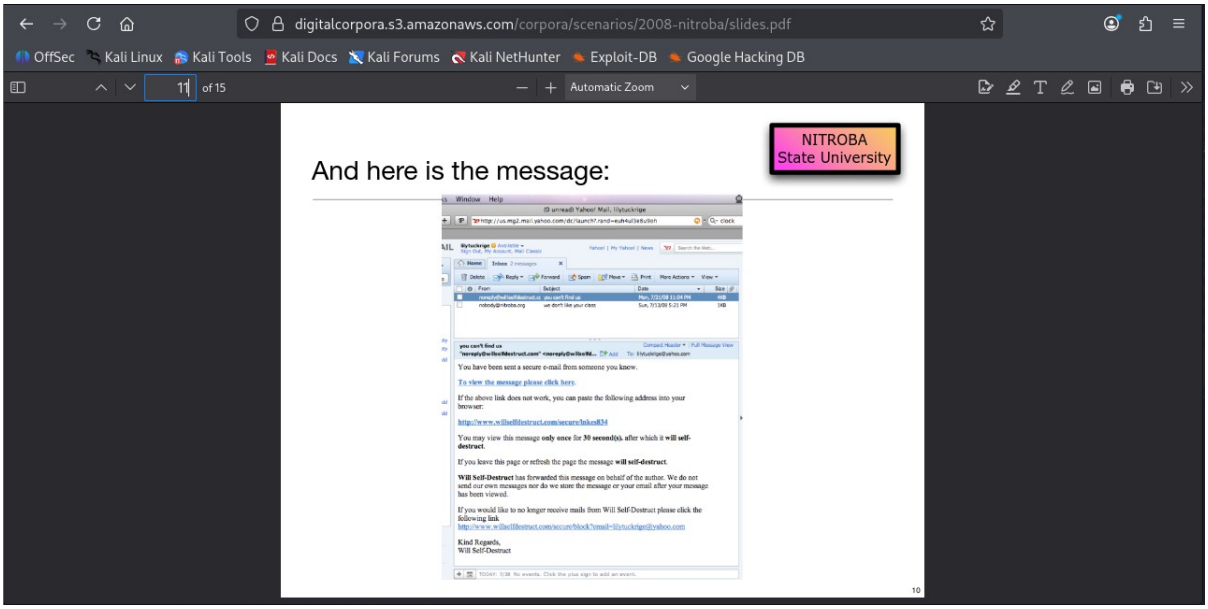


Figure 13: Scenario slide (PDF page 11): notification email stating the message can be viewed only once for 30 second(s). (E14)

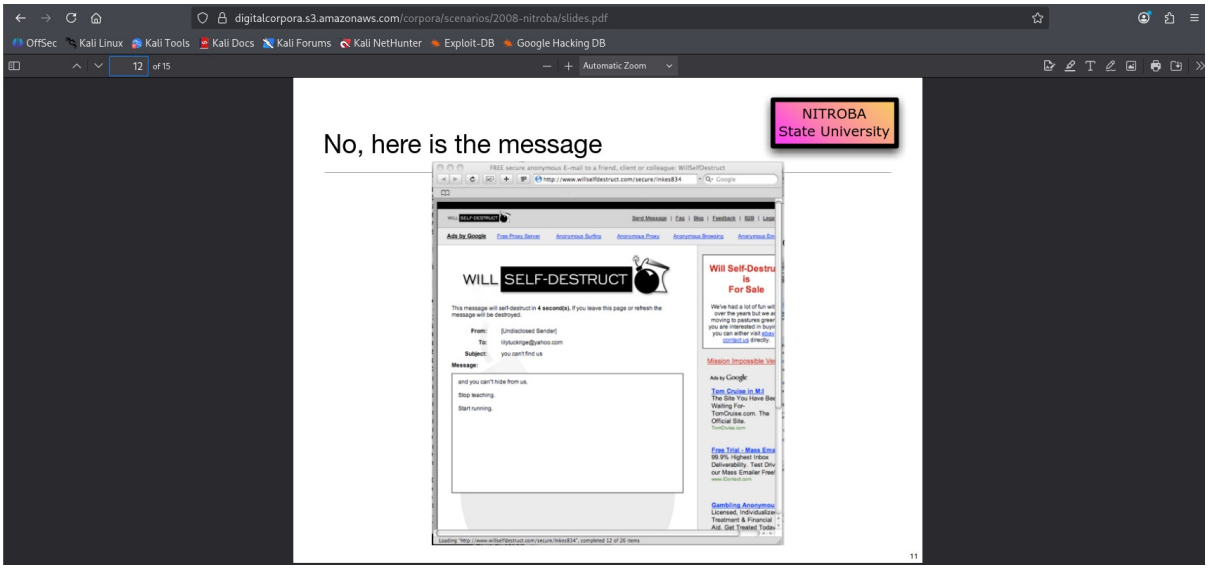


Figure 14: Scenario slide (PDF page 12): the message as displayed by willselfdestruct.com, from [Undisclosed Sender] to lilytuckrige@yahoo.com. (E14)

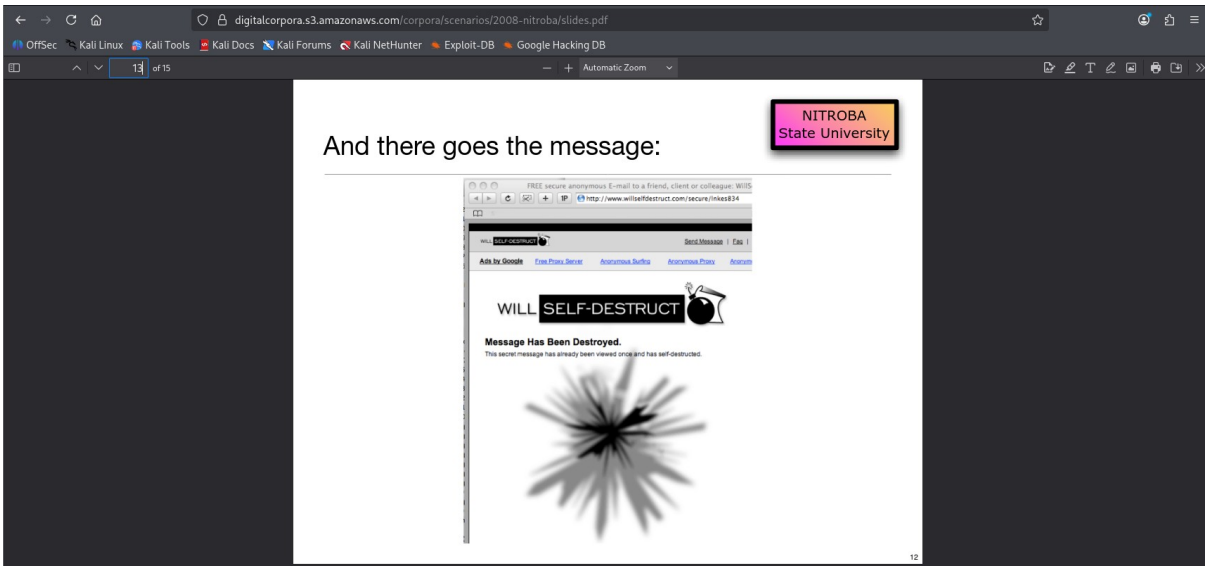


Figure 15: Scenario slide (PDF page 13): "Message Has Been Destroyed" page. (E14)

5.4 Question 4: Which device made the relevant request?

Display filter / command	Packet / stream	Timestamp (UTC)	Evidence ID	Figures
frame 82936 and 83601, Ethernet II layer; eth.src == 00:17:f2:e2:c0:ce && http.request.method == "POST"	Packets 82936, 83601	06:03:43.825871; 06:04:24.311700	E07	Fig 16, 17
tshark -T fields -e eth.src -e ip.src (MAC/IP inventory); ip.src == 192.168.15.4 -> eth.src; eth.src == MAC && not ip	34,554 frames; 28 ARP frames	Whole capture	E08	Fig 18, 19

Direct observation: the Ethernet source address of both the GET (82936) and the POST (83601) is 00:17:f2:e2:c0:ce (Wireshark's vendor lookup labels it Apple_e2:c0:ce). The destination MAC is 00:1d:d9:2e:4f:60. The two packets share Ethernet stream index 19. The MAC inventory shows 34,554 frames from this MAC, all with source IP 192.168.15.4; the remaining 28 frames from this MAC are ARP and carry no IP source. Every one of the 34,554 frames from 192.168.15.4 came from

this one MAC. The MAC also sent 362 HTTP POST requests in total, mostly Google mail and calendar traffic.

Inference: the destination MAC 00:1d:d9:2e:4f:60 is the local gateway. It appears as the source of frames with IP 192.168.15.1 and of frames carrying many external source IPs, which is typical of a router forwarding replies to the client. The vendor labels are from Wireshark's OUI lookup and are not proof of the device make.

Why an IP address alone cannot identify the sender. 192.168.15.4 is a private address assigned behind the room's router; anyone who joins the network can obtain such an address, and addresses can be reassigned. The scenario slides state that the router had no password (PDF page 7), so any nearby device could connect. The MAC is stronger evidence than the IP because it points to a particular network card, which here matches a single IP over the whole capture. It still identifies a device and not a person: several people can use one computer, and MAC addresses can be changed (spoofed). The email header IP (140.247.62.34) is a public address that the slides resolve to G24.student.nitroba.org (PDF page 6); I did not test whether that address appears in the capture, so the NAT link between the two is an inference from the scenario description.

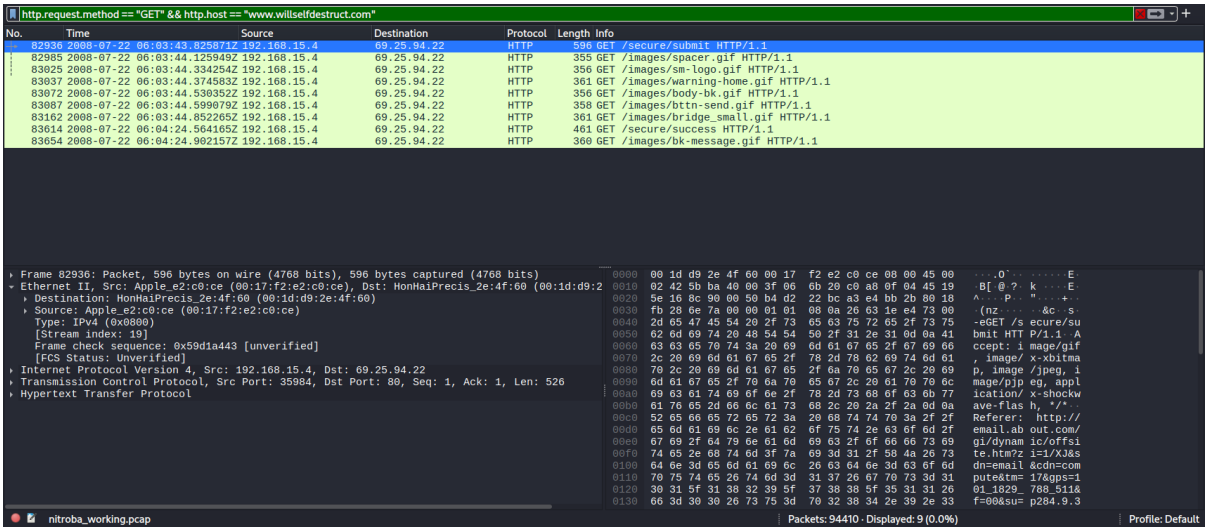


Figure 16: Ethernet II layer of packet 82936 (GET /secure/submit): source MAC 00:17:f2:e2:c0:ce, destination 00:1d:d9:2e:4f:60. (E07)

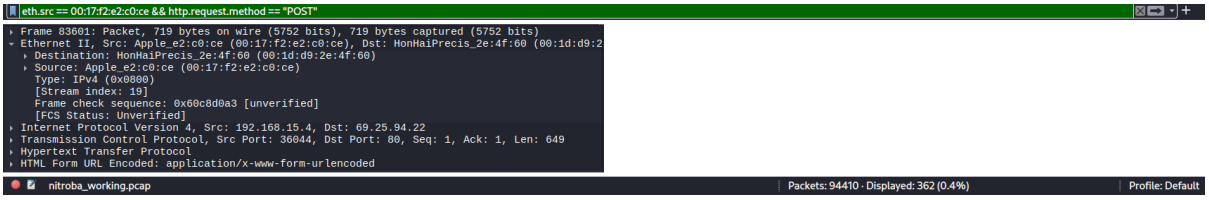


Figure 17: Ethernet II layer of packet 83601 (the POST): same source MAC. Only the filter bar, details pane and status bar are shown; the packet list was cropped to avoid unrelated session data. (E07)



Figure 18: MAC/IP inventory of the capture (top 20 rows): 34,554 frames from 00:17:f2:e2:c0:ce with source IP 192.168.15.4; 2,154 frames from 00:1d:d9:2e:4f:60 with source IP 192.168.15.1. (E08)



Figure 19: Non-IP frames from the client MAC (28, all ARP) and MAC check for 192.168.15.4 (34,554 frames, one MAC only). (E08)

5.5 Question 5: Who is associated with the device?

Display filter / command	Packet / stream	Timestamp (UTC)	Evidence ID	Figures
eth.src == 00:17:f2:e2:c0:ce && http.cookie contains "@" (149 packets); http.cookie contains "gmailchat=jcoachj" (100 packets)	Packets 78967 to 84201; nearest 83326 (before) and 84201 (after)	06:01:02.113436 to 06:04:55.380722	E09, E10	Fig 20, 21
User-Agent comparison; willselfdestruct requests with a Cookie header	10 willselfdestruct requests; 100 Gmail requests	06:01:02 to 06:04:55	E11	Fig 22

Direct observation: 149 packets from the client MAC carry a cookie containing “@”. Most are advertising or tracking values that match “@” by coincidence and have no identity value. Two cookie fragments name an account: gmailchat=jcoachj@gmail.com/475090 (100 packets, all to mail.google.com at 74.125.19.17) and OL_SESSION=jcoachj@gmail.com-cal (9 requests). The first gmailchat packet is 78967 (06:01:02.113436 UTC), and the last is 84201 (06:04:55.380722 UTC). The nearest cookie-bearing packets to the POST are 83326, 18.77 s before it (06:04:05.546193), and 84201, 31.07 s after it. So Gmail activity from this device brackets the willselfdestruct session (06:03:43 to 06:04:25 UTC).

Direct observation: the User-Agent on the Gmail requests (97 mail.google.com, 2 b.mail.google.com, 1 chatenabled.mail.google.com) and on the ten willselfdestruct.com requests is the same: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1). A check for any willselfdestruct request carrying a Cookie header returned no rows, so no cookie ties the willselfdestruct request itself to the account.

Inference: the same browser on the same device used the jcoachj@gmail.com account and submitted the willselfdestruct form within a minute of each other. The shared User-Agent is consistent with this but is weak evidence, because MSIE 6.0 on Windows NT 5.1 was extremely common in 2008. The cookie evidence shows that this account was in use on the device; it does not show who typed the message. Raw cookie output contained unrelated tracking and session tokens and is withheld; only the identity-bearing fragments are reproduced.



Figure 20: Packets carrying gmailchat=jcoachj: 100 in total (first three and last three shown) and those nearest the POST (83326 and 84201), all to mail.google.com (74.125.19.17). (E09)

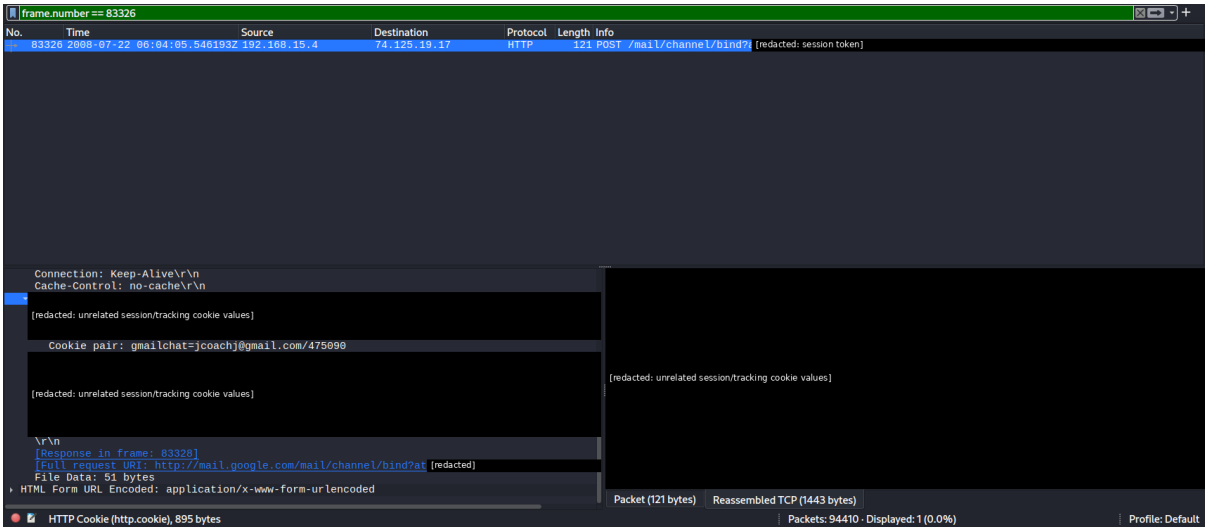


Figure 21: Packet 83326 (06:04:05.546193 UTC) with the Cookie header expanded. Only the gmailchat cookie pair is shown; all other cookie values and token fields are redacted. (E09)



Figure 22: User-Agent comparison for the client MAC (identical string on Gmail and willselfdestruct requests) and the check that no willselfdestruct request carries a Cookie header (no output). (E11)

5.6 Question 6: Is that person on the Chem 109 roster?

Display filter / command	Packet / stream	Timestamp (UTC)	Evidence ID	Figures
Roster from scenario slides (PDF page 14); cookie fragment from E09	Packets 78967 to 84201	06:01:02 to 06:04:55	E12	Fig 23

The Chemistry 109 class list in the scenario slides contains eleven names and no email addresses: Amy Smith, Burt Greedom, Tuck Gorge, Ava Book, Johnny Coach, Jeremy Ledvkin, Nancy Colburne, Tamara Perkins, Esther Pringle, Asar Misrad and Jenny Kant. The account name in the cookie is jcoachj@gmail.com. The only roster entry whose surname matches the “coach” element of that account name is Johnny Coach.

This is a name-pattern inference and not a documented match. The roster has no email addresses, the trailing “j” in the account name is not explained by the roster, and anyone could create an account resembling a student’s name. The correct statement is that the account in use on the device is consistent with the roster entry Johnny Coach, not that Johnny Coach is proven to be the sender.

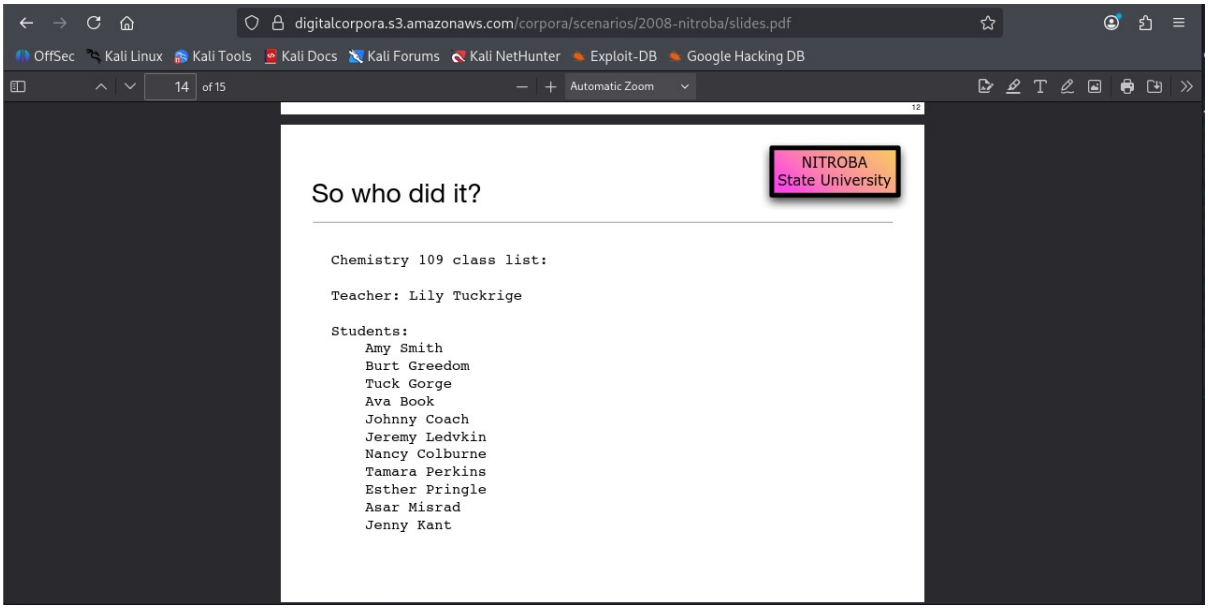


Figure 23: Scenario slide (PDF page 14): Chemistry 109 class list (names only). (E12)

5.7 Question 7: When did the activity occur?

See Section 6 for the full timeline. In summary, the form was requested at 06:03:43.825871 UTC, the POST was sent 40.49 s later at 06:04:24.311700 UTC, and the server redirected to the success page 0.157 s after that (06:04:24.469169 UTC). All times are UTC from the PCAP. The time zone displayed by Wireshark is UTC after the display format was set; before that, Wireshark showed the VM's local time, EDT (UTC-4). The only conversion made is EDT to UTC (+4 h), and the Yahoo mailbox time to UTC (+7 h, under the assumption stated in Section 5.3).

5.8 Question 8: What conclusion can you defend?

The defensible conclusion is the one in Section 2. The capture supports attribution of the willselfdestruct message to a device (MAC 00:17:f2:e2:c0:ce, IP 192.168.15.4) in the shared dorm room, and shows that an account matching the Chem 109 roster entry Johnny Coach was in use on that device at the time. It does not establish that the account holder personally sent the message. The detailed assessment, confidence levels and limitations are in Section 7.

6. Timeline

Times from the PCAP are UTC, from Wireshark's UTC display. Context rows from the scenario slides are marked and are not from the capture. EDT = Kali VM local time (UTC-4).

Time (UTC)	Packet	Event	Source
2008-07-14 00:21:01	n/a	First harassing email received (header: Sun, 13 Jul 2008 17:21:01 -0700; Received from 140.247.62.34)	Slides p.5 (context)
2008-07-22 01:51:07.095278	1	Capture begins (21:51:07 EDT on 21 July in VM local time)	capinfos
2008-07-22 06:01:02.113436	78967	First request carrying gmailchat=jcoachj@gmail.com cookie (to mail.google.com)	E09
06:03:43.825871	82936	GET /secure/submit to www.willselfdestruct.com (192.168.15.4 to 69.25.94.22)	E03
06:03:44.125949 to 06:03:44.852265	82985, 83025, 83037, 83072, 83087, 83162	Six GETs for form-page images (spacer.gif, sm-logo.gif, warning-home.gif, body-bk.gif, bttn-send.gif, bridge_small.gif)	E03

Time (UTC)	Packet	Event	Source
06:04:05.546193	83326	Gmail channel POST carrying the gmailchat cookie (18.77 s before the POST)	E09
06:04:24.311700	83601	POST /secure/submit: to=lilytuckrige@yahoo.com, subject “you can’t find us”, ttl=30	E04
06:04:24.469169	83604	HTTP/1.1 302 Moved Temporarily, Location /secure/success (server Date header reads 07:24:45 GMT)	E05, E16
06:04:24.564165	83614	GET /secure/success	E03
06:04:24.902157	83654	GET /images/bk-message.gif	E03
06:04:55.380722	84201	Last request carrying the gmailchat cookie (31.07 s after the POST)	E09
2008-07-22 06:13:47.046029	last	Capture ends (duration 15,759.95 s)	capinfos
2008-07-22 06:04 (inferred)	n/a	Second email shown in the Yahoo inbox as Mon 7/21/08 11:04 PM (mailbox clock assumed UTC-7)	Slides p.10 (context)

7. Attribution Assessment

7.1 Observed facts

- Device MAC 00:17:f2:e2:c0:ce with IP 192.168.15.4 made all nine GETs and the only POST to www.willselfdestruct.com (69.25.94.22) (E03, E04, E06, E07).
- The POST’s recipient, subject, message text and 30-second lifetime match the second email in the scenario slides (E04, E14).
- That MAC appears with only one IP address, and that IP with only one MAC, across the whole capture; the MAC’s other 28 frames are ARP (E08).
- 100 requests from the same MAC carried gmailchat=jcoachj@gmail.com, from 06:01:02.113436 to 06:04:55.380722 UTC, both before and after the POST (E09).
- The User-Agent was identical across the Gmail and willselfdestruct requests, and no willselfdestruct request carried a Cookie header (E11).
- The roster lists names only (E12). The slides state that three women share the room and that the Wi-Fi router had no password (E15).

7.2 Reasonable inferences

- One browser on one device both used the jcoachj Gmail account and submitted the form, based on matching MAC, IP, User-Agent and timing.
- Packet 83601 is the harassing message delivered as the second email (matching recipient, subject, text, lifetime and time).
- The account name jcoachj corresponds to the roster entry Johnny Coach (name-pattern only).
- The private address 192.168.15.4 sits behind the same router as the email-header IP 140.247.62.34 (scenario description; not tested in the capture).

7.3 Confidence

Proposition	Confidence	Basis
This device (MAC 00:17:f2:e2:c0:ce) sent the willselfdestruct message	Moderate to high	Direct packets; one POST; content matches the second email; device uniquely mapped to one IP
The jcoachj account holder was using that device	Moderate	100 cookie-bearing requests before and after the POST; no cookie on the willselfdestruct request itself
The jcoachj account corresponds to roster entry Johnny Coach	Low to moderate	Name-pattern match; roster has no emails
That person personally typed and sent the message	Low to moderate	Shared open network, shared room and shared device possible; no direct authentication

7.4 Shared open wireless network limitation

The slides state that three women share the room (Alice, Barbara, Candice), that Nitroba provides Ethernet but no Wi-Fi, that Barbara's boyfriend Kenny installed a Wi-Fi router, and that the router had no password (PDF page 7). A network like this cannot bind an IP address to a single person: any resident, visitor or neighbour in range could join and be given a similar private address. The sniffer placement (PDF page 8) shows several devices connecting through the router before the capture point, so traffic from the room is a mix of devices. This is why the report relies on the MAC address for the device, and on the cookie for the account, instead of the IP address.

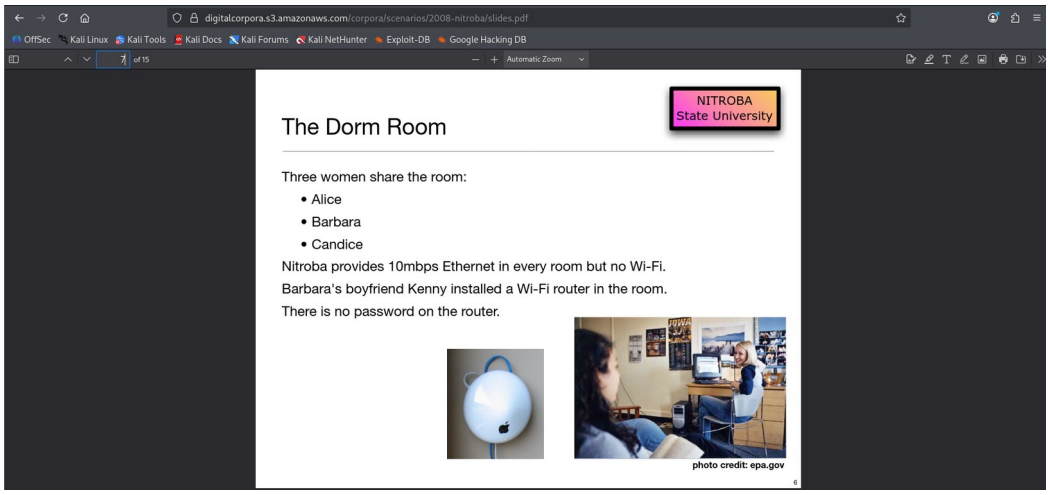


Figure 24: Scenario slide (PDF page 7): three residents share the room; Wi-Fi router installed with no password. (E15)

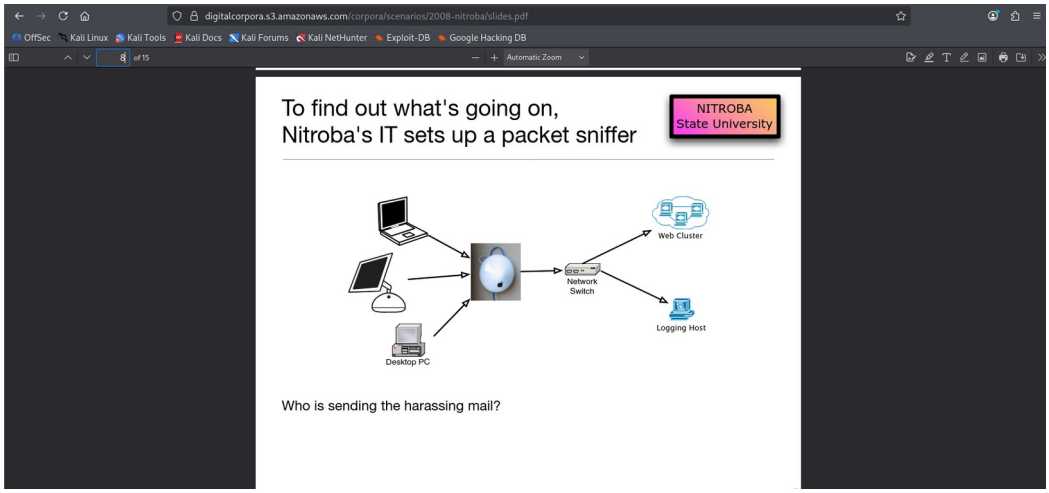


Figure 25: Scenario slide (PDF page 8): devices connect through the router to a network switch with a logging host. (E15)

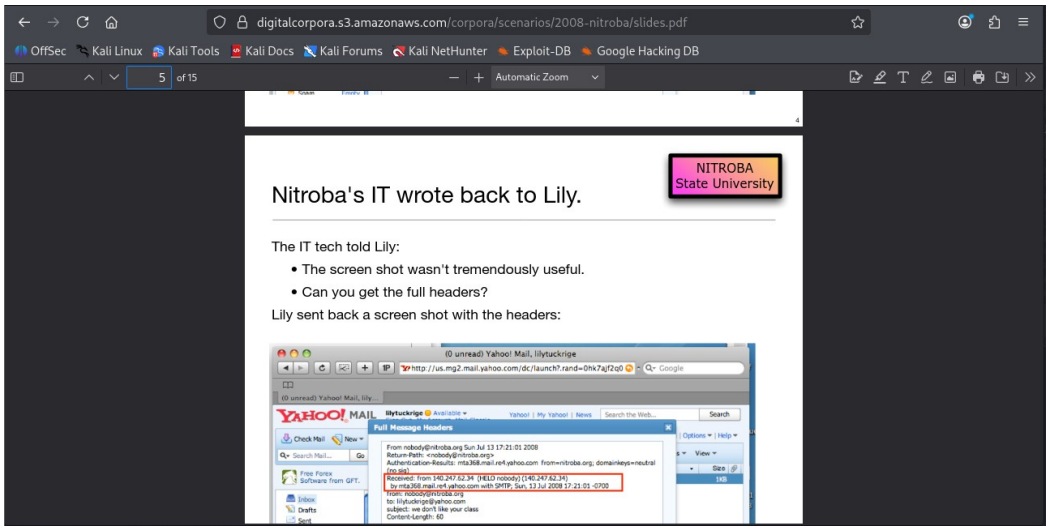


Figure 26: Scenario slide (PDF page 5): full headers of the first email, with the Received line from 140.247.62.34 highlighted (Sun, 13 Jul 2008 17:21:01 -0700). (E13)

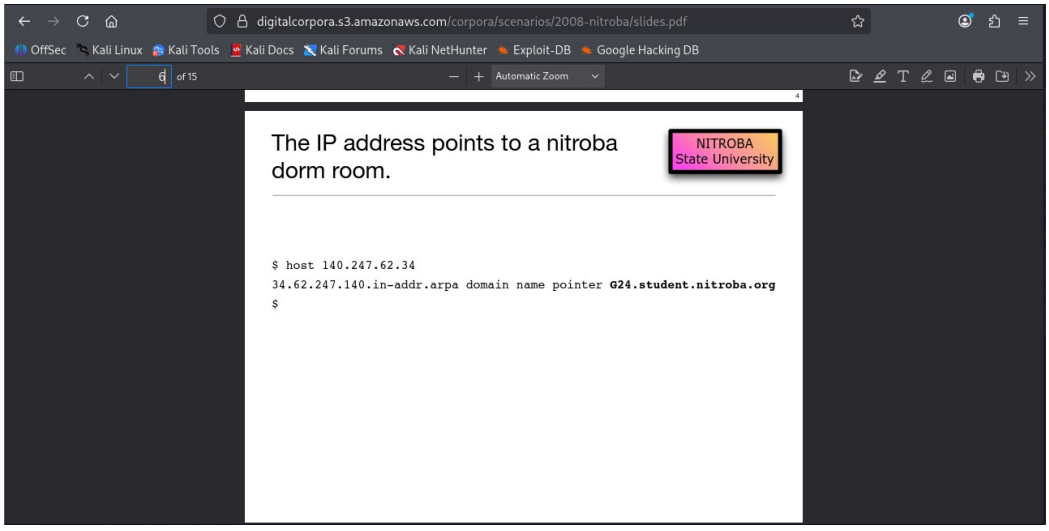


Figure 27: Scenario slide (PDF page 6): host 140.247.62.34 resolves to G24.student.nitroba.org. (E13)

7.5 Limitations and alternative explanations

- Shared access: another resident or visitor could have used the same computer, or the same browser profile, while the jcoachj account was logged in.

- Open Wi-Fi and MAC spoofing: a different device could connect to the unsecured router. MAC addresses can be changed, so the MAC identifies a device only on the assumption that it was not spoofed. The capture shows one MAC per IP, but a spoofed MAC would look the same.
- No identity at the service: the form's from field is blank, the slide shows an undisclosed sender, and no cookie accompanied the willselfdestruct request, so the account link is by timing and device only.
- Account ownership: the cookie shows the account was in use, not who owned it. The mapping to a roster name is by name pattern, and the roster has no email addresses.
- Clock discrepancies: the willselfdestruct server's Date header (07:24:45 GMT) differs from the capture time of the same reply (06:04:24 UTC) by about 1 h 20 min, so the web server's clock appears to be wrong. The Yahoo mailbox time zone (UTC-7) is assumed. The scenario page also states that its timestamps are not relevant to solving the problem, so absolute times should be treated with caution and relative timing inside the capture is the stronger evidence.
- Single capture point and slides: the capture shows only traffic crossing one point, and the emails, headers and roster exist only as slide screenshots, not as original message files.
- Weak browser fingerprint: MSIE 6.0 on Windows NT 5.1 was extremely common in 2008, so the User-Agent match does not single out a user.

Remaining uncertainty: the evidence points strongly to one device and to an account consistent with one roster entry, but it cannot show who was operating the device. A stronger case would need independent evidence such as login records for the jcoachj account, the router's association logs, or witness evidence of who used the room's computers at 06:04 UTC.

Appendix A: Evidence Log

Evidence ID	Packet or item	Finding	Why it matters	Screenshot or appendix
E01	nitroba.pcap file	56,180,821 bytes; SHA-256 2b77a9ea…67553ec2fb; MD5 and SHA-1 also recorded; all match published values	Identifies the exact file analysed and proves integrity	Fig 1, 3, 4
E02	nitroba_working.pcap; capinfos	Working copy hash identical; original read-only; 94,410 packets; span 01:51:07.095278 to 06:13:47.046029 UTC	Shows analysis was on a verified copy; fixes UTC time basis	Fig 2, 5
E03	Packet 82936 (and 8 other GETs)	GET /secure/submit, 192.168.15.4 to 69.25.94.22, Host www.willselfdestruct.com, 06:03:43.825871 UTC, stream 1680	Identifies client IP, service IP and start of the web session	Fig 6, 7
E04	Packet 83601; stream 1707	POST /secure/submit; to lilytuckrige@yahoo.com; subject “you can’t find us”; message text; ttl=30; 06:04:24.311700 UTC	Links the client to the harassment message	Fig 8, 9, 10
E05	Packets 83604, 83614	302 Moved Temporarily to /secure/success at 06:04:24.469169 UTC; client then GETs /secure/success (06:04:24.564165 UTC)	Shows the submission was accepted by the service	Fig 9, 10
E06	post_packets.csv (tshark export)	Exactly one POST to willselfdestruct.com in the capture	No other client posted to the service	Fig 11
E07	Packets 82936, 83601 (Ethernet II)	Source MAC 00:17:f2:e2:c0:ce on both; destination 00:1d:d9:2e:4f:60	Identifies the device that made the requests	Fig 16, 17
E08	MAC/IP inventory; ARP check	MAC maps only to 192.168.15.4 (34,554 frames) plus 28 ARP; IP uses only this MAC	Supports device identification and its limits	Fig 18, 19
E09	Packets 78967 to 84201 (100); packet 83326	gmailchat=jcoachj@gmail.com/475090 cookie to mail.google.com; 83326 at 06:04:05.546193 and 84201 at 06:04:55.380722 bracket the POST	Associates the device with an account	Fig 20, 21
E10	Cookie fragments (filtered)	gmailchat=jcoachj@gmail.com/475090 (100); OL_SESSION=jcoachj@gmail.com-cal (9); raw dump withheld	Confirms the account name in two cookies	evidence_records / cookie_identity_fragments.txt
E11	User-Agent check; cookie check on willselfdestruct requests	Same MSIE 6.0/Windows NT 5.1 User-Agent on Gmail and willselfdestruct requests; no Cookie header on willselfdestruct requests	Supports same-browser inference; shows account link is not direct	Fig 22
E12	Slides PDF page 14	Class list of 11 names, no emails; only Johnny Coach matches “coach”	Roster comparison and its limits	Fig 23
E13	Slides PDF pages 5, 6	First email header: Received from 140.247.62.34, 17:21:01 -0700; IP resolves to G24.student.nitroba.org	Context for the shared dorm IP; supports the UTC-7 mailbox assumption	Fig 26, 27

Evidence ID	Packet or item	Finding	Why it matters	Screenshot or appendix
E14	Slides PDF pages 10 to 13	Second email in inbox (Mon 7/21/08 11:04 PM), notification (30 s), message text, “Message Has Been Destroyed”	Matches the POST to the reported message	Fig 12 to 15
E15	Slides PDF pages 7, 8	Three residents; Wi-Fi router with no password; sniffer behind router	Basis of the shared-network limitation	Fig 24, 25
E16	Packet 83604 Date header	Server Date Tue, 22 Jul 2008 07:24:45 GMT vs capture 06:04:24.469169 UTC	Shows a clock discrepancy; limits use of absolute times	Fig 9

Appendix B: Command Log

Commands run on Kali Linux in ~/SBTDF204_CS1 (working copy: working/nitroba_working.pcap).

```
# Acquisition and integrity
wget https://digitalcorpora.s3.amazonaws.com/corpora/scenarios/2008-nitroba/nitroba.pcap
date -u ; stat nitroba.pcap ; ls -l nitroba.pcap
sha256sum nitroba.pcap ; md5sum nitroba.pcap ; chmod a-w nitroba.pcap
cp -p nitroba.pcap ../working/nitroba_working.pcap ; sha256sum nitroba_working.pcap ;
capinfos nitroba_working.pcap
TZ=UTC capinfos -a -e nitroba_working.pcap
tshark -r nitroba_working.pcap -c 1 -T fields -e frame.number -e frame.time_epoch -e
frame.time_utc
echo "<published sha256>  ../original/nitroba.pcap" | sha256sum -c -   # likewise md5sum -c
and sha1sum -c

# Exports and checks
tshark -r working/nitroba_working.pcap -Y 'http.request.method == "POST" && http.host
contains "willselfdestruct"' -T fields -e frame.number -e frame.time_utc -e
frame.time_epoch -e eth.src -e eth.dst -e ip.src -e ip.dst -e tcp.srcport -e tcp.stream -e
http.request.uri -E header=y -E separator=,
tshark -r working/nitroba_working.pcap -T fields -e eth.src -e ip.src | sort | uniq -c |
sort -rn | head -20
tshark -r working/nitroba_working.pcap -Y 'eth.src == 00:17:f2:e2:c0:ce && not ip' -T
fields -e _ws.col.protocol | sort | uniq -c
tshark -r working/nitroba_working.pcap -Y 'ip.src == 192.168.15.4' -T fields -e eth.src |
sort | uniq -c
tshark -r working/nitroba_working.pcap -Y 'eth.src == 00:17:f2:e2:c0:ce && http.cookie
contains "gmailchat=jcoachj"' -T fields -e frame.number -e frame.time_utc -e ip.dst -e
http.host
tshark -r working/nitroba_working.pcap -Y 'http.host contains "willselfdestruct" &&
http.cookie' -T fields -e frame.number -e http.cookie | head
tshark -r working/nitroba_working.pcap -Y 'eth.src == 00:17:f2:e2:c0:ce && (http.host
contains "willselfdestruct" || http.cookie contains "gmailchat=jcoachj")' -T fields -e
http.host -e http.user_agent | sort | uniq -c | sort -rn
```

Appendix C: Redaction and Handling Notes

- Screenshots were cropped to the Wireshark, terminal or PDF-viewer window; the VM frame and host desktop were removed.
- The translucent terminal window showed faint text from other windows behind it. Terminal screenshots were processed to blank the dim background so that only the terminal text remains. The terminal text itself was not altered.

- Figure 17: the packet list was cropped out because other rows in that list show Gmail session tokens in the Info column.
- Figure 21: all cookie values other than the gmailchat pair, the hex pane and the token in the request URI and Info column are covered with black boxes and labelled as redacted.
- The raw cookie listing was not included because it contains unrelated tracking and session values. Only the identity-bearing fragments are reproduced.
- The original PCAP is not included in this submission, in line with the brief.
- The scenario slides contain fictional names and addresses; they are reproduced only as needed to support the findings.